Manual de usuario

Sistema de reserva de turnos | GrupoIIS2026 | Parte 6
Versión 1.0 · 6 de noviembre de 2026

Este manual explica cómo administrar establecimientos y personal, solicitar y cancelar reservas y consultar agenda y disponibilidad. La versión entregada se opera mediante una API REST, Swagger UI, scripts y MQTT; no incluye una interfaz de pantallas propia para clientes finales.

1. Acceso al sistema

El sistema debe estar iniciado según el manual de instalación. Abrir en el navegador:

```
http://localhost:8080/api/swagger-ui.html
```

En Swagger, expandir la operación deseada, pulsar Try it out, completar los parámetros y el cuerpo JSON cuando corresponda, y pulsar Execute. Revisar Server response y Response body. Los grupos o textos pueden aparecer con nombres generados de los controladores.

Las rutas del manual se agregan a http://localhost:8080/api. Por ejemplo, GET /establecimientos se invoca en http://localhost:8080/api/establecimientos. La definición revisada de la entrega incluye más descripciones y errores que la generada automáticamente en Swagger.

2. Cómo interpretar las respuestas

Código HTTP	Significado para el usuario
200	Consulta o actualización completada.
201	Establecimiento o empleado creado; guardar el id recibido.
202	Solicitud de reserva o cancelación aceptada para procesar. Consultar después su estado.
204	Eliminación completada; no hay cuerpo de respuesta.
400	Datos, JSON, parámetros o formatos inválidos.
404	No existe el recurso solicitado o relacionado.
409	Eliminación impedida por personal o reservas asociados.
503	No se pudo conectar o publicar en Mosquitto.
500	Error interno; revisar registros con el responsable del sistema.

Los ejemplos usan IDs y fechas ilustrativos. Se deben sustituir por los IDs devueltos por la aplicación y por un día y horario adecuados para la prueba. Para la demostración se usan atenciones de 30 minutos y fechas sin zona horaria.

Administrar establecimientos

3. Registrar un local

Ejecutar POST /establecimientos con este cuerpo JSON:

```
{
  "nombreComercial": "Local de prueba",
  "direccion": "Calle de prueba 123",
  "telefono": "099123456",
  "correo": "local@example.com",
  "horarioApertura": "08:00:00",
  "horarioCierre": "18:00:00"
}
```

Todos los campos son obligatorios. El correo debe tener un formato válido y la apertura debe ser anterior al cierre. El teléfono es texto para conservar ceros iniciales. La respuesta HTTP 201 contiene el local y su id: conservar ese identificador para asociar empleados.

4. Consultar, modificar o eliminar

Operación	Uso
GET /establecimientos	Listar los locales.
GET /establecimientos/{id}	Consultar un local reemplazando {id} por su identificador.
PUT /establecimientos/{id}	Actualizar usando el cuerpo completo de registro con los nuevos valores.
DELETE /establecimientos/{id}	Eliminar el local si no tiene personal asociado.

Ejemplo: para cambiar el nombre, usar PUT /establecimientos/1 y enviar todos los campos, con el nuevo nombreComercial. No se debe enviar únicamente el campo cambiado. Si el local tiene empleados, DELETE devuelve 409: primero se deben resolver esas asociaciones.

5. Organización del personal

Cada empleado se asocia a un establecimiento. El horario de atención del empleado se toma del local; no se configura una agenda horaria independiente por empleado en esta versión. Mantener los horarios del establecimiento coherentes con los turnos que se desean ofrecer.

Consultar los empleados del local:

```
GET /establecimientos/1/personal
```

La respuesta contiene una lista; si no tiene empleados, es una lista vacía. Si el establecimiento no existe, se responde 404.

Administrar personal

6. Registrar un empleado

Usar POST /personal y enviar un establecimientoId existente:

```
{
  "nombre": "Profesional de prueba",
  "especialidad": "Atención general",
  "costoConsulta": 1500,
  "duracionEstandar": 30,
  "estado": true,
  "establecimientoId": 1
}
```

nombre y especialidad no pueden estar vacíos. costoConsulta es un entero no negativo; duracionEstandar debe ser 30. estado es booleano: true activo, false inactivo. Las altas y actualizaciones CRUD admiten ambos estados; un empleado inactivo no puede atender nuevas reservas. La respuesta 201 incluye el id del empleado.

Alternativa: POST /establecimientos/1/personal toma el local desde la ruta. En este caso enviar el mismo JSON sin establecimientoId.

7. Operaciones disponibles

Operación	Uso
GET /personal	Listar todo el personal.
GET /personal/{id}	Consultar un empleado.
POST /personal	Crear personal indicando el local en el cuerpo.
POST /establecimientos/{id}/personal	Crear personal en el local indicado en la ruta.
GET /establecimientos/{id}/personal	Listar empleados de ese local.
PUT /personal/{id}	Modificar datos completos del empleado.
DELETE /personal/{id}	Eliminar personal sin reservas asociadas.

Para cambiar el costo o desactivar personal, usar PUT con el objeto completo y los nuevos valores. Si tiene reservas asociadas, incluso históricas o canceladas, DELETE devuelve 409. Desactivar mediante estado=false permite conservar el historial sin eliminar el empleado.

Antes de solicitar un turno, guardar el id del establecimiento y del personal. El empleado debe pertenecer al establecimiento solicitado.

Solicitar y cancelar reservas

8. Solicitar un turno

Usar POST /reservas con los datos del cliente, del establecimiento, del empleado y del inicio del turno:

```
{
  "emailSolicitante": "cliente@example.com",
  "telefonoSolicitante": "099123456",
  "establecimientoId": 1,
  "personalId": 2,
  "fechaHoraTurno": "2026-12-10T11:00:00"
}
```

Todos los campos son obligatorios y los IDs deben ser positivos. La fecha se escribe como año-mes-día, T y hora local, sin Z ni offset. El ejemplo significa 10 de diciembre a las 11:00. Reemplazar IDs y fecha al ejecutar la operación.

La API devuelve 202 con un id de reserva y estado SOLICITADO. Conservar el id y consultar GET /reservas/{id}. El procesador revisa periódicamente (cada 60 segundos en Compose) las reglas y puede agendar o rechazar. Un 202 no garantiza que el horario esté reservado.

9. Interpretar el estado de la reserva

Estado	Significado
SOLICITADO	Solicitud registrada, pendiente de evaluación.
AGENDADO	Solicitud aceptada y horario asignado.
RECHAZADO_SOLICITUD_NO_VALIDA	Datos o condiciones de negocio no válidos. Revisar registros del procesador para el motivo.
RECHAZADO_TURNO_OCUPADO	El horario se solapa con un turno que impide asignarlo.
CANCELADO	Reserva cancelada; se conserva el registro.
ATENDIDO	El proceso automático determinó que ya terminó el horario del turno. No acredita asistencia real.
FACTURADO	La reserva fue incorporada a la facturación.

10. Cancelar un turno

Usar DELETE /reservas/{id}. La API devuelve 202 y un mensaje de aceptación; el listener procesa la solicitud. Consultar GET /reservas/{id} hasta comprobar CANCELADO. Se cancelan reservas SOLICITADO o AGENDADO. Repetir una cancelación ya realizada no modifica el resultado; no se cancelan reservas atendidas o facturadas por esta operación. No confundir este DELETE con eliminación física del registro.

Consultas y facturación

11. Consultar reservas y disponibilidad

Operación	Resultado
GET /reservas	Todas las reservas y sus estados.
GET /reservas/{id}	Una reserva y su estado actual.
GET /personal/{id}/reservas	Historial de reservas asignadas al empleado.
GET /personal/{id}/reservas?fecha=2026-12-10	Reservas del empleado para ese día.
GET /personal/{id}/disponibilidad?fechaHora=2026-12-10T11:00:00	Objeto con personalId, fechaHoraTurno y disponible.

En Swagger, completar id y los parámetros fecha o fechaHora en sus campos, sin escribir la cadena completa de la URL. La consulta HTTP de reservas del empleado incluye estados históricos y cancelados. La agenda MQTT actual filtra AGENDADO, ATENDIDO y FACTURADO, por lo que sus resultados pueden diferir.

disponible=true indica disponibilidad en ese momento; no bloquea el horario. false puede indicar personal inactivo, duración no válida, horario fuera de atención o solapamiento. Un empleado inexistente genera 404. Para reservar, realizar después POST /reservas y observar su estado.

12. Facturación automática

El procesador marca AGENDADO como ATENDIDO cuando se cumple su hora de finalización, y publica un aviso. Cada 300 segundos el facturador revisa turnos atendidos y los agrupa por cliente, año y mes; registra cliente, factura e ítem y cambia la reserva a FACTURADO. El cálculo usa los datos de costo de consulta del producto. No se requiere una llamada manual para iniciar cada ciclo.

Esta versión no expone endpoints REST de clientes ni facturas, ni emite un comprobante fiscal. Para revisar los registros de facturación se utiliza MySQL Workbench o el script consultar-base.ps1. Se verificaron las reservas 7 y 9 en FACTURADO, con factura e ítem de 1500.00 cada una, en la base de demostración original.

13. Ejemplo de error HTTP

```
{
  "fechaHora": "2026-11-06T12:00:00",
  "estado": 404,
  "error": "Not Found",
  "mensaje": "No existe una reserva con ID 999",
  "ruta": "/api/reservas/999"
}
```

Leer mensaje para identificar el problema. Los errores de validación pueden indicar varios campos. Un error asíncrono de negocio puede verse como estado rechazado de la reserva, aunque la solicitud inicial haya devuelto 202.

Mensajería y demostración

14. Observar mensajes MQTT

Desde PowerShell, en la raíz del proyecto:

```
docker compose exec -T mosquitto mosquitto_sub -h localhost -t "turnos/#" -v
```

La terminal queda escuchando; Ctrl+C la detiene y devuelve el prompt. Utilizar otra terminal para publicar mensajes mientras se observa. También puede conectarse MQTT Explorer a localhost, puerto 1883. La demostración no requiere usuario ni contraseña MQTT.

Para consultar mediante MQTT, usar los contratos de Especificacion-MQTT.pdf. turnos/agenda y turnos/disponibilidad reciben consultas; sus respuestas se publican en los mismos nombres terminados en /respuesta. Suscribirse antes de publicar. Las altas y cancelaciones no generan una respuesta MQTT de confirmación: verificar sus resultados en API o base.

15. Ejecutar los scripts de demostración

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\parte6\establecimientos.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\parte6\personal.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\parte6\reservas.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\parte6\consultar-base.ps1
```

Los primeros scripts realizan operaciones reales con datos de prueba. establecimientos.ps1 y personal.ps1 crean, consultan, actualizan y eliminan sus registros de demostración. reservas.ps1 crea un local y un empleado, solicita un turno, espera AGENDADO, consulta agenda y disponibilidad, cancela y comprueba que el horario queda libre. Conserva los registros de esa demostración. consultar-base.ps1 muestra los datos de las seis tablas.

16. Recomendaciones para una demostración

Usar IDs devueltos por el sistema, no asumir que siempre serán 1 o 2. Para evitar que las reservas aleatorias interfieran con una prueba manual, el responsable puede detener el generador con docker compose stop app-generador y reanudarlo con docker compose start app-generador. Los otros procesos pueden seguir funcionando.

Antes de cerrar una prueba, verificar el estado final en lugar de basarse solamente en el envío del comando. No intentar eliminar un empleado con historial de reservas para limpiar una demostración; su registro puede conservarse o desactivarse. La eliminación de volúmenes corresponde a administración y borra la base: no forma parte del uso normal del producto.

Documentos complementarios: Manual-instalacion.pdf, MER-final.pdf, Arquitectura-componentes.pdf, especificacion-api.json y Especificacion-MQTT.pdf. Las rutas HTTP, los campos y las condiciones descritas se basan en el código proporcionado y en las pruebas guiadas realizadas por el equipo.